

UNIVERSIDAD

Ciencias de la Computación VIII

PROYECTO SERVIDOR ASÍNCRONO

Visualización progresiva de imágenes gigantes con carga selectiva por tiles

Estudiante: Samantha Rodas

Lenguaje del servidor: Java 21

Protocolo de aplicación: IRP/1.0 - Image Resolution Protocol

Fecha: Septiembre 2026

1. Introducción

El proyecto implementa un servidor asíncrono en Java para visualizar imágenes de alta resolución sin transferir la imagen completa al navegador. La solución divide cada imagen en niveles de resolución y fragmentos (tiles), de modo que el cliente solicita únicamente los fragmentos que intersectan el área visible. Cuando el usuario cambia de nivel, el cliente libera los recursos del nivel anterior y solicita los nuevos tiles necesarios.

La comunicación inicial del sitio se realiza mediante HTTP/1.1. Después, el navegador establece una conexión WebSocket con el servidor y sobre ella se ejecuta el protocolo propio IRP/1.0. Todos los recursos del frontend y las imágenes son atendidos localmente por el servidor Java; no se requieren servicios externos ni conexión a Internet.

2. Objetivos

2.1 Objetivo general

Desarrollar un servidor asíncrono multicliente capaz de servir imágenes de alta resolución de forma progresiva y selectiva, reduciendo transferencia innecesaria, uso de memoria y carga del navegador.

2.2 Objetivos específicos

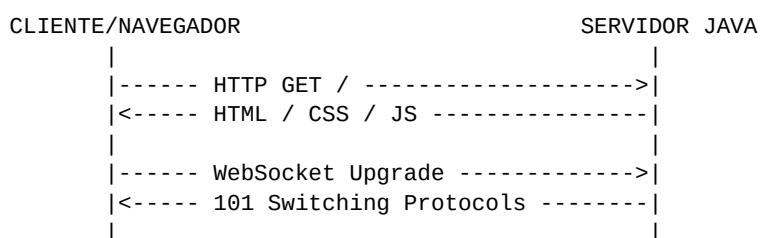
- Atender múltiples clientes mediante canales asíncronos de Java.
- Servir HTML, CSS y JavaScript por HTTP desde el propio servidor.
- Generar automáticamente una pirámide multiresolución para imágenes normales y gigantes, incluyendo PSB y TIFF.
- Dividir cada nivel de resolución en tiles JPEG de 512 x 512 píxeles para reducir el número de archivos y solicitudes en imágenes de decenas de gigabytes.
- Transferir únicamente los tiles visibles para la vista actual del cliente.
- Definir y documentar un protocolo propio para listar imágenes, consultar metadatos y solicitar tiles.
- Liberar del lado del cliente los tiles que dejan de ser necesarios al cambiar de resolución.

3. Arquitectura de la solución

La solución se divide en cuatro componentes principales:

- Servidor HTTP asíncrono: recibe conexiones, sirve el frontend y realiza el Upgrade a WebSocket.
- Administrador de imágenes: detecta el formato/tamaño, reutiliza cache existente y selecciona procesamiento Java o procesamiento de disco para imágenes gigantes.
- Protocolo IRP/1.0: controla listado, metadatos, nivel y selección de tiles.
- Frontend: dibuja los tiles sobre un canvas, calcula el área visible y administra el cache del navegador.

Flujo general



```
|----- IRP/1.0 LIST ----->|
|<----- lista de imágenes -----|
|----- IRP/1.0 INFO ----->|
|<----- dimensiones/niveles -----|
|----- IRP/1.0 TILE ----->|
|<----- cabecera IRP + JPEG binario ----|
```

4. Implementación del servidor

4.1 Asincronía y concurrencia

El servidor utiliza `AsynchronousServerSocketChannel`. Las operaciones de aceptación, lectura y escritura se completan mediante `CompletionHandler`. Este enfoque evita mantener un hilo bloqueado por cada conexión. Las tareas de acceso a archivos y manejo de solicitudes HTTP se ejecutan en un `ExecutorService` para no bloquear el flujo de I/O asíncrono.

4.2 Procesamiento de imágenes

Al iniciar el servidor se recorren imágenes JPG, JPEG, PNG, TIFF/TIF y PSB almacenadas en `images/originals`. Las imágenes pequeñas pueden procesarse con `Java ImageIO`. Los archivos PSB/TIFF, los archivos mayores a 200 MB o las imágenes con dimensiones muy grandes se envían a un procesador de disco controlado por Java. Ese procesador invoca `ImageMagick` localmente y genera una pirámide de resolución dividida en tiles JPEG de 512 x 512 píxeles. De esta forma Java no necesita construir en memoria un `BufferedImage` del raster gigante completo. La preparación se realiza una sola vez y queda cacheada en `images/processed`.

4.2.1 Imágenes gigantes y PSB

La imagen de referencia `eso1242a` del Observatorio Europeo Austral tiene 108199 x 81503 píxeles y un archivo original de aproximadamente 24.6 GB. Para un caso de este tamaño no es viable utilizar `ImageIO.read()` sobre el archivo completo. La versión final detecta PSB/TIFF, archivos mayores a 200 MB o raster con dimensiones muy grandes y activa el procesamiento externo en disco.

`ImageMagick` se ejecuta como proceso local iniciado desde Java. Se limita el uso de memoria y mapa, se utiliza almacenamiento temporal en disco y cada nivel terminado queda marcado con un archivo `.complete`. Si el procesamiento se interrumpe, los niveles terminados se conservan. El `metadata.json` registra tamaño y fecha del archivo fuente para evitar reprocesamientos innecesarios.

Con tiles de 512 x 512, la imagen `eso1242a` produce aproximadamente nueve niveles desde una vista reducida hasta 108199 x 81503. El servidor nunca envía el PSB al navegador; solamente lee del disco y transmite los JPEG correspondientes al viewport actual.

Ejemplo de estructura generada:

```
images/processed/eso1242a/
  metadata.json
  level0/0_0.jpg
  level1/0_0.jpg ...
  ...
  level8/25_14.jpg ...
```

4.3 Servidor de archivos iniciales

El navegador obtiene index.html, style.css y app.js mediante solicitudes HTTP GET. El servidor determina el Content-Type del archivo solicitado y devuelve Content-Length y Connection: close. No se utilizan CDN, fuentes remotas ni bibliotecas externas.

5. Protocolo IRP/1.0

IRP/1.0 (Image Resolution Protocol) es el protocolo de aplicación definido para controlar la transferencia de imágenes. WebSocket se utiliza como transporte compatible con navegadores; los comandos, campos, códigos y reglas de selección de resolución pertenecen a IRP/1.0.

Comando	Función	Campos principales
HELLO	Verifica la comunicación con el protocolo	-
LIST	Obtiene las imágenes disponibles	-
INFO	Consulta dimensiones, niveles y tile size	Image
TILE	Solicita un fragmento de un nivel	Image, Level, X, Y
CLOSE	Finaliza lógicamente la sesión IRP	-

Ejemplo de solicitud TILE:

IRP/1.0 TILE

Image: eso1242a

Level: 8

X: 25

Y: 14

Respuesta de control:

IRP/1.0 200 OK

Content-Type: image/jpeg

Content-Length: 42117

Image: eso1242a

Level: 8

X: 25

Y: 14

[frame binario JPEG]

Códigos implementados:

- 200 OK - operación válida.
- 400 BAD REQUEST - comando/protocolo inválido.
- 404 NOT FOUND - imagen o tile inexistente.
- 409 INVALID REQUEST - nivel o parámetros inválidos.
- 500 INTERNAL ERROR - error no controlado del servidor.

6. Gestión del cliente y uso de memoria

El frontend utiliza un canvas HTML para reconstruir la zona visible con tiles independientes. Antes de solicitar datos calcula los índices X/Y que intersectan el viewport. Cada tile solicitado se registra como pendiente para evitar solicitudes duplicadas. Los tiles recibidos se convierten en ImageBitmap y se almacenan temporalmente en un Map.

Cuando el usuario cambia de nivel de resolución, el cliente ejecuta close() sobre cada ImageBitmap, vacía el cache y solicita únicamente los tiles visibles del nuevo nivel. Por lo tanto, el navegador no mantiene simultáneamente todas las resoluciones ni recibe la imagen original completa.

7. Pruebas realizadas

Antes de empaquetar la entrega se verificaron las siguientes condiciones:

- Compilación completa con javac usando los archivos del directorio src.
- Respuesta HTTP 200 para la ruta principal del sitio.
- Handshake WebSocket correcto en /irp.
- Comando IRP LIST con respuesta 200 OK.
- Comando IRP INFO con respuesta 200 OK y metadatos de la imagen demo.
- Envío consecutivo de múltiples comandos TILE y recepción de sus correspondientes frames JPEG binarios.
- Procesamiento automático de una imagen demo y validación del flujo PSB mediante un archivo PSB de prueba. La arquitectura para PSB gigante evita cargar el raster completo en Java y almacena la pirámide en disco.

Prueba multicliente sugerida para la demostración:

1. Ejecutar make run.
2. Abrir http://localhost:8080 en dos o más pestañas o navegadores.
3. Mover cada vista de manera independiente y cambiar niveles de resolución.
4. Observar en la consola del servidor solicitudes HTTP e IRP de los clientes.
5. Abrir DevTools > Network > WS para observar los mensajes IRP.
6. Desactivar Internet y verificar que la aplicación continúa funcionando localmente.

8. Estructura del proyecto

```
ProyectoServidorAsincrono/  
  Makefile  
  README.md  
  src/  
    Main.java  
    image/ImageManager.java  
    image/ImageMetadata.java  
    protocol/IrpProtocol.java  
    server/AsyncHttpServer.java  
    server/WebSocketUtil.java  
  web/  
    index.html  
    style.css  
    app.js  
  images/originals/  
  images/processed/  
  docs/
```

9. Ejecución

En Linux o WSL:

Para imágenes gigantes, primero ejecutar:
make prepare

Una vez creada la pirámide cacheada, iniciar el servidor con:
make run

También puede ejecutarse manualmente:

```
mkdir -p out
javac -encoding UTF-8 -d out $(find src -name "*.java")
java -cp out Main -port 8080
```

Luego abrir <http://localhost:8080> en el navegador.

10. Conclusiones

La implementación evita el envío de la imagen completa y reemplaza esa estrategia por una transferencia selectiva basada en resolución y zona visible. Para PSB/TIFF gigantes, el servidor Java controla un procesamiento local con ImageMagick que genera una pirámide cacheada en disco, sin construir el raster completo en memoria Java. Durante la navegación el cliente conserva únicamente los tiles requeridos y libera los anteriores al cambiar de nivel. El uso de I/O asíncrono permite atender múltiples conexiones sin aplicar el modelo tradicional de un hilo bloqueado por cliente.

11. Referencias

- IETF. RFC 9110: HTTP Semantics. 2022.
- IETF. RFC 9112: HTTP/1.1. 2022.
- IETF. RFC 6455: The WebSocket Protocol. 2011.
- Oracle. Java Platform, Standard Edition 21 - `java.nio.channels.AsynchronousServerSocketChannel`.
- Oracle. Java Platform, Standard Edition 21 - `javax.imageio.ImageIO` y `java.awt.image.BufferedImage`.
ImageMagick. Image Formats - PSB, Adobe Large Document Format.
ESO. eso1242a - VISTA gigapixel mosaic, 108199 x 81503 px, original 24.6 GB.